

Hand-On-Learning: Deploy and Monitor Your First LangChain Service

Learning Objectives

By the end of this activity, learners will be able to:

- Containerize a Python-based LangChain microservice using a production-minded Dockerfile.
- Push a built image to **AWS ECR** and reference it from Kubernetes manifests.



- Create deployment.yaml and service.yaml files to deploy a microservice on Kubernetes with basic health configuration.
- Add **OpenTelemetry** instrumentation and expose a Prometheus-ready metrics endpoint.
- Interpret basic telemetry (latency, throughput, error rate) to identify performance bottlenecks.



Description

In this hands-on practice, you'll act as a backend engineer at **FinQuery**, turning a local LangChain document analysis script into a **production-ready, observable service**. You'll package the service into a Docker image, publish it to **AWS ECR**, deploy it on **Kubernetes**, and wire up **OpenTelemetry + Prometheus** so your ops team can monitor it in real time as it scales from 200 to 2,000+ documents per day.

Structure of the Activity

Title

Deploy and Monitor Your First LangChain Service

Dataset information / Resources Required

Application code:

finquery_analyzer.py – Python LangChain service that processes regulatory filings (local-only script).

Configuration stubs (optional):

Example requirements.txt or pyproject.toml

Sample YAML templates (minimal Kubernetes deployment/service skeletons)

Tools used:

→ Containerization & Registry:

→ Docker

→ AWS ECR

→ Orchestration:

→ Kubernetes cluster (local or cloud)

→ kubectl CLI

Observability:

- OpenTelemetry SDK for Python
- Prometheus
- (Optional) Grafana for visualization

Setup Requirements

- Docker installed and running.
- Access to a Kubernetes cluster (e.g., minikube, kind, or managed K8s).
- AWS CLI configured with credentials and permissions to use ECR.
- Python 3.10+ installed locally (for building/running the app when needed).
- `finquery_analyzer.py` available in the working directory.

Scenario

You're a developer at **FinQuery**, a financial services startup that extracts insights from regulatory filings using a LangChain-based analysis microservice. Right now, the service is just a **local Python script**, processing about 200 documents per day.

A new **enterprise pilot** changes everything: a major client will send over **2,000+ documents daily starting next week**. Your CTO has set three non-negotiable requirements before launch:

1. The service must run on **Kubernetes** with automatic scaling potential.
2. The container image must be stored in **AWS ECR**.
3. The service must include **OpenTelemetry + Prometheus** instrumentation so operations can see latency, throughput, and error rates in real time.

Currently, there is:

- No Dockerfile
- No Kubernetes manifests
- No telemetry or metrics

You have **five days** in the scenario—but **25 minutes** in this lab—to produce a first production-ready version.

Objective

Transform the local finquery_analyzer.py LangChain script into a **containerized, deployed, and observable** microservice running on Kubernetes with metrics exposed for Prometheus.



Instructions for Participants

Getting Started

- 1 Open `finquery_analyzer.py` in your IDE and confirm you understand its main entrypoint (how requests are handled).
- 2 Ensure you can run the script locally with a sample document to verify it works.
- 3 Create a project folder (for example: `finquery-deploy/`) to hold the Dockerfile and Kubernetes manifests.

Activity 1: Containerize the FinQuery Service with Docker

Objective

Create a production-ready Docker image for the LangChain-based analyzer and push it to **AWS ECR**.



Input/Data

→ finquery_analyzer.py

→ requirements.txt (or equivalent dependency definition)

Steps to Complete

Step 1: Write the Dockerfile

→ In the project folder, create a Dockerfile that:

- Document Uses a slim Python base image (e.g., python:3.11-slim).
- Sets a working directory (e.g., /app).
- Copies in requirements.txt and installs dependencies using pip with pinned versions.
- Copies in finquery_analyzer.py (and any supporting modules).
- Creates a non-root user and runs the app as that user.

Defines a clear CMD or ENTRYPOINT to start the service (e.g., uvicorn or a custom server script).

Step 2: Build the Image Locally

- Run `docker build -t finquery-analyzer:1.0 .` (or similar tag).
- Verify the image is created with `docker images`.

Step 3: Tag & Push to AWS ECR

- Create (or identify) an ECR repository for finquery-analyzer.
- Authenticate Docker to ECR using AWS CLI.
- Tag the local image with the full ECR URI (e.g., ACCOUNT_ID.dkr.ecr.REGION.amazonaws.com/finquery-analyzer:1.0).
- Push the image to ECR and confirm it appears in the console.

Expected Outcome:

A Docker image that can run `finquery_analyzer.py` in any environment, stored in AWS ECR and ready for Kubernetes deployment.

Tips:

- Do **not** bake secrets (API keys) into the image; use environment variables.
- Use `.dockerignore` to avoid copying `.git` and large local files.

Activity 2: Deploy the Service on Kubernetes

Objective

Create Kubernetes manifests to deploy the FinQuery analyzer image and expose it via a stable service endpoint.



Input/Data

- Image URI from AWS ECR (Activity 1).
- Expected listening port of your service (e.g., 8000 or 8080).

Steps to Complete

Step 1: Create deployment.yaml

→ Define a Deployment resource that:

→ References the ECR image.

→ Sets replicas: 1 initially.

→ Configures necessary environment variables (e.g., OPENAI_API_KEY)

→ via env or envFrom (but not hard-coded secrets).

→ Includes basic readinessProbe and livenessProbe (e.g., hitting /healthz or a simple gRPC health check if applicable).

Step 2: Create service.yaml

Define a Service resource that:

- Selects pods via labels from the Deployment (e.g., app: finquery-analyzer).

- Exposes the service as ClusterIP or LoadBalancer depending on your cluster.

- Maps from a service port (e.g., 80) to the container port.

Step 3: Apply Manifests and Verify

- Run `kubectl apply -f deployment.yaml` and `kubectl apply -f service.yaml`.
- Confirm pods are running: `kubectl get pods`.
- Check status/details with `kubectl describe deployment finquery-analyzer` and `kubectl get svc`.

Expected Outcome:

A running FinQuery analyzer deployment on Kubernetes, reachable via a Service with basic health checks configured.

Tips:

- Keep labels consistent between Deployment and Service.
- If probes fail, temporarily relax them to ensure the pod can start, then refine.

Activity 3: Instrument with OpenTelemetry & Expose Prometheus Metrics

Objective

Add basic **OpenTelemetry** instrumentation to the service and expose a **Prometheus** metrics endpoint to support real-time monitoring.



Input/Data

- Running Kubernetes deployment from Activity 2.
- Access to Prometheus (cluster-integrated or a simple deployment).

Steps to Complete

Step 1: Add OpenTelemetry Instrumentation

→ Modify `finquery_analyzer.py` (or a separate `app.py`) to:

- Initialize an OpenTelemetry tracer provider early in the application startup.
- Create spans around key operations: fetching documents, calling the LLM, generating summaries.
- Configure an exporter to send traces to an OpenTelemetry Collector or a local receiver, if available.

Step 2: Expose a Metrics Endpoint

Use an OpenTelemetry metrics SDK or a simple library (e.g., `prometheus_client`) to:

- Define counters/histograms for request count and latency.

- Expose a `/metrics` HTTP endpoint.

Step 3: Configure Prometheus

- Ensure Prometheus is configured to scrape the service's `/metrics` endpoint (e.g., via a scrape config that matches the service label).

- Validate that metrics are visible in Prometheus (`/graph` UI or similar).

Step 4: Generate Load and Inspect Metrics/Traces

- Send several test requests (simulate the regulatory filings workload).
- Observe:
 - Request counts increasing.
 - Latency distribution (p50/p95, if available).
 - Error rates, if any.
- Optionally, visualize metrics in a Grafana dashboard.

Expected Outcome:

Evidence (screenshot or export) showing **live metrics and/or traces** for the FinQuery LangChain service under test load.

Tips:

Start with minimal instrumentation: key spans and 1–2 core metrics (request count + latency).

Focus on signal quality, not perfect dashboards—this lab is about wiring the basics.

Compile Your Findings

Requirement 1: Containerization & Deployment Assets

Question Prompt:

"Upload the folder or repository that contains your Dockerfile, deployment.yaml, and service.yaml. How do these assets demonstrate a production-minded containerization and Kubernetes deployment for the FinQuery analyzer service?"

Response Type:

File Upload

File Types:

Coding, Text, Audio, Video

→ .py, .yaml, .yml, .txt, .md, .json

→ .mp3, .wav, .aac

→ .mp4, .mov, .webm

Requirement 2: Observability Evidence

Question Prompt:

"Upload a screenshot or export of your Prometheus or Grafana dashboard showing metrics or traces from your deployed FinQuery service. What key performance signals does your screenshot illustrate?"

Response Type:

File Upload

File Types:

Image, Video, Text

→ .png, .jpg, .jpeg, .webp, .tiff

→ .mp4, .mov, .webm

→ .txt, .md, .pdf

Requirement 3: Observability Reflection

Question Prompt:

"In 150–250 words, describe which metrics or traces provided the most insight into system performance and where you identified bottlenecks. How would your observability setup help FinQuery operate reliably at 2,000+ documents per day?"

Response Type:

Rich Text

File Types:

Not applicable (Rich Text submission)

Requirement 4: Limitations & Future Improvements

Question Prompt:

""What limitations or risks exist in your current deployment and observability setup, and what next steps would you recommend to improve resilience, performance, or monitoring?"

Response Type:

File Upload

File Types:

Text, Audio, Video

→ .txt, .md, .pdf

→ .mp3, .wav, .aac

→ .mp4, .mov, .webm

Summary

This activity provides hands-on practice in **deploying and monitoring a LangChain service in a realistic production context**. You've:

- Turned a local Python script into a **Dockerized microservice** stored in **AWS ECR**.
- Deployed it to **Kubernetes** with basic health checks and a stable network endpoint.
- Instrumented it with **OpenTelemetry** and exposed metrics for **Prometheus**, enabling real-time monitoring.

Key takeaways:

- Containerization and Kubernetes deployment turn “works on my machine” into “works in the cluster.”
- Observability—traces and metrics—is essential to understanding performance and bottlenecks as load grows.
- Even simple telemetry (request count, latency, error rate) can reveal whether your LangChain service is truly ready for a 10× traffic increase.

Resources

Primary Materials:

- finquery_analyzer.py starter file
- Your Dockerfile, deployment.yaml, and service.yaml

Reference Documentation:

- Docker docs (image build & ECR push)
- Kubernetes docs for Deployments and Services
- OpenTelemetry Python docs
- Prometheus scraping configuration

Additional Resources (Optional):

- grpcurl documentation or Postman gRPC guide
- Example LangChain + gRPC sample projects (if provided in the course repo)

Share The Project

Document: Save the report in PDF format.

Upload: Share in the “Peer Review” area of the course shell with a brief description.

Instructions for Documenting and Sharing The Project for Peer Review

1. Document the Project

- Use word processing software to create your report.
- Ensure all sections of the project document structure are completed.
- Proofread and edit your report for clarity and accuracy.

2. Share the Project

- Save your report in PDF format.
- Upload your documents to the “Peer Review” area in the course shell.
- Provide a brief description of your project in the submission post.

Instructions for Documenting and Sharing The Project for Peer Review

3. Peer Review

- Review the projects submitted by your peers.
- Provide constructive feedback and comments on their work.